ENGINEERING / SIMULATION REPORT / V1.0

20轴伺服同步控制与安全机制技术报告

20-Axis Servo Control, Synchronization & Safety Simulation

基于 C++17 的异构关节动力学、离散PID、五次多项式轨迹、通信延迟、故障注入、心跳与看门狗验证

20	1 ms	4
异构关节模型	仿真控制步长	已运行工况

文档性质： 机器人控制软件学习与仿真实验报告
证据来源： 项目源码、CSV日志、事件记录、实测调度数据
编制日期： 2026年10月28日　　**重要声明：** 非实物试验、非安全认证

OVERVIEW

01 摘要与结论

本报告针对一套 **20轴异构伺服控制软件仿真**，说明动力学建模、控制算法、通信延迟、统一时间基准以及安全监督机制的实现方式，并以实际生成的CSV和事件日志评价正常工况与三类故障工况。项目采用C++17，仿真周期1 ms，单次实验总时长4.5 s，关节总数20。所有关节在同一时刻采样各自的五次多项式参考轨迹，但使用独立的PID状态、惯量、阻尼和执行器限制。

20/20	0.000406	128 ms
正常工况稳定到位	最大跟踪误差 / rad	实际到位时间跨度

故障注入验证了三种不同触发源：第8轴机械卡滞引起持续跟踪误差，第13轴的心跳丢失，以及控制器心跳丢失。三种故障都触发软件级组停机并最终进入STOPPED标签。仿真显示故障被发现并非瞬时，例如卡滞注入至触发保护相隔356 ms；这一延迟应该在后续开发中纳入安全需求，而不能忽略。

> **关键限制**
>
> 报告中的“安全停止”仅表示仿真程序施加的速度阻尼制动，不是经独立硬件实现的SS1、SS2、STO或急停。1 ms的仿真步长也不代表操作系统或现场总线具备硬实时1 kHz能力。

报告依据与可信度边界

证据	内容	结论边界
源码	`src/main.cpp` 中的模型、PID、DelayLine和监督器	证实“实现了什么”，不证实物理设备表现
日志	4组实验的CSV、事件CSV和summary文件	证实本次给定参数下的模拟行为
调度测量	正常工况以 `--realtime` 运行的唤醒时间数据	仅代表当时运行环境；不是硬实时认证
文献资料	ROS实时设计、Linux计时器、EtherCAT DC、驱动安全功能文档	用于解释设计原则，不等于本项目已采用相关硬件

目录

01 摘要与结论
02 需求、指标和实验边界
03 软件架构与数据流
04 20轴关节动力学建模
05 五次轨迹与运动同步
06 离散PID和执行器处理
07 命令延迟与统一时钟
08 心跳、看门狗与状态机
09 故障注入与实验设计
10 实验结果与数据分析
11 调度抖动和实时性
12 代码结构与复现说明
13 设计缺口和工程升级路线
14 测试清单、面试总结与参考资料

REQUIREMENTS

02 需求、指标和实验边界

该项目的设计目标不是精确复现某一款工业机器人，而是构建可控、可观测、可复现的多轴控制平台，使面试中常见的“电流异常、关节卡住、通信掉线、多个电机同步、看门狗如何工作”等问题具有能够测试的工程答案。

目标	实现手段	验证手段
多轴动力学差异	20组不同J、D、摩擦、转矩/速度/位置限制	检查配置函数、记录关节响应

同步轨迹规划	所有关节共享0.25 s起点和3.0 s时长	比较参考轨迹完成时刻与实测到位时刻
闭环控制	位置误差、速度误差、模型前馈与抗积分饱和	观察各轴跟踪误差和实际转矩
延迟模拟	每轴独立命令延迟队列和序列号	检查延迟逻辑及异常包处理
软件安全监督	心跳、命令更新、反馈合法性、限位、持续跟踪误差	三类故障注入、触发原因和时间
调度性能	steady_clock与sleep_until定时	统计唤醒迟到量和相邻周期偏差

验收口径

正常工况要求20轴进入位置与速度容差；故障工况要求记录注入时刻、发现时刻、保护原因和减速完成时刻。另需明确区别：①参考轨迹同步；②实际稳定到位同步；③真实驱动器时钟同步。三者不等价。

实验的主要假设

- 各轴动力学相互独立；未考虑多关节耦合、重力、柔性减速器和真实接触碰撞。
- 关节参数由代码人为指定，不是通过真实电机辨识得到；文中J/D曲线属于配置可视化。
- 驱动器收到控制命令后按照限幅和转矩斜率模型执行；电流、逆变器PWM和真实编码器没有建模。
- 通信层是软件离散延迟模型，不是经过EtherCAT或CAN实物测试的总线时延。

ARCHITECTURE

03　软件架构与控制数据流

项目使用一个统一的仿真主循环，按每个1 ms tick推进20个关节的状态；没有20个独立运行且相互竞争的控制线程。这样能清楚地分离“共享时间基准”与“每轴动力学不同”两个概念，适合先验证同步控制算法。



一个控制周期的执行顺序

1. 依据当前tick计算虚拟时间，若开启实时测量则等待绝对计划唤醒时刻并记录唤醒迟到量。
2. 按工况更新心跳，注入指定故障；对全部关节在相同t采样轨迹。
3. 处于RUNNING时，逐轴计算PID力矩指令，并送入各自的延迟队列。
4. 消费已经到达的有效命令；检查是否出现命令更新超时。

5. 运行软件监督器：心跳、反馈有效性、位置/速度限位、持续跟踪误差。
6. 在RUNNING或模拟制动模式下积分动力学，并记录位置、速度、转矩和安全状态。

> **重要区别**
> 监督器虽然在逻辑上独立于PID决策，但当前版本仍位于同一进程、同一主循环中，不能防护进程整体挂死。控制器“冻结”工况仅通过停止心跳与更新命令进行建模，安全监督循环仍会执行；这与真实CPU死机或电源失效不同。

PLANT MODEL

04　20轴异构关节动力学模型

4.1 连续动力学

单轴采用等效刚性转动惯量、黏性阻尼和光滑库仑摩擦模型：

$$J_i\,\ddot{q}_i = \tau_i - D_i\,\dot{q}_i - \tau_{c,i}\tanh(\dot{q}_i / 0.025)$$

其中J是等效惯量，D是黏性阻尼系数，τ_c是库仑摩擦系数。tanh使零速附近的摩擦函数平滑，便于数值积分，但不能精确描述真实电机的静摩擦、Stribeck效应、齿隙和滞回。

4.2 执行器限制和数值积分

模型先对期望转矩做幅值限幅，再按照每步最大转矩变化量限制转矩斜率。采用半隐式Euler更新速度与位置：

$$\Delta\tau_i = \mathrm{clamp}(\tau_{req} - \tau_{applied},\ -S_i T_s,\ S_i T_s)$$
$$\dot{q}[k+1] = \dot{q}[k] + T_s \cdot \ddot{q}[k] \qquad q[k+1] = q[k] + T_s \cdot \dot{q}[k+1]$$

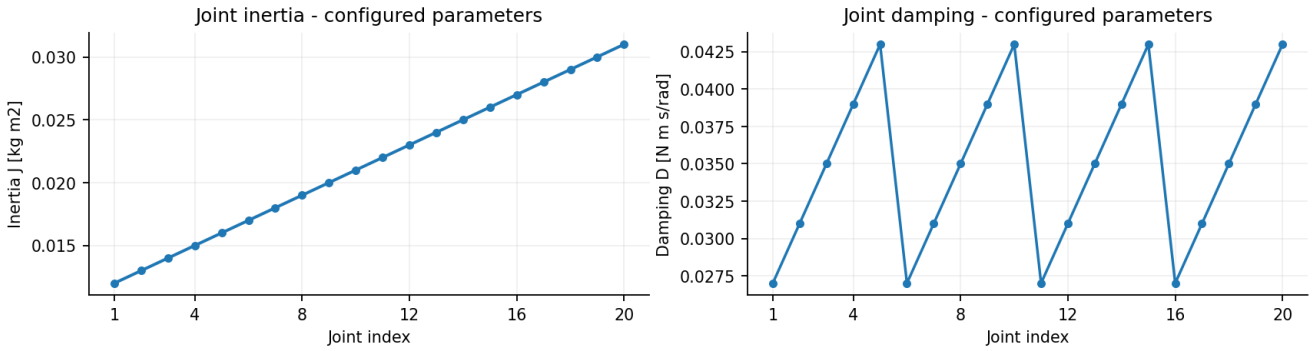


图4-1：20个关节的配置惯量与阻尼差异。两组参数由make_config()确定，均为合成参数，并非辨识结果。

参数	代码取值范围	单位
等效惯量 J	0.012–0.031	kg · m²
黏性阻尼 D	0.027–0.043	N · m · s/rad
库仑摩擦系数 τ_c	0.0025–0.0040	N · m
转矩上限	0.70–1.03	N · m
转矩斜率上限	20–26	N · m/s
速度约束	1.00–1.24	rad/s
位置约束	±1.7	rad

位置、速度限制在该模拟器中是安全监督阈值，而不是积分函数中的强制截断；若越界，由安全逻辑检测。严格的执行器模型还需要电流、供电、电压饱和、温度和制动器等参数。

PLANNING

05 五次多项式轨迹和多轴同步

5.1 共同时间轴

所有关节在0.25 s开始运动，轨迹持续3.0 s，因此期望轨迹的统一结束时刻是3.25 s。各关节目标角度不同、正负交替，目标位移绝对值从0.20 rad逐步增加到0.637 rad。

$$r = (t - 0.25) / 3.0 \qquad s(r) = 10r^3 - 15r^4 + 6r^5$$
$$q_{d,i}(t) = q_{0,i} + \Delta q_i s(r)$$

速度和加速度均通过对s(r)求导获得，控制器直接接收q_d、v_d、a_d。五次多项式满足轨迹两端速度、加速度为零，适合在静止起止的场景进行平滑时间参数化。

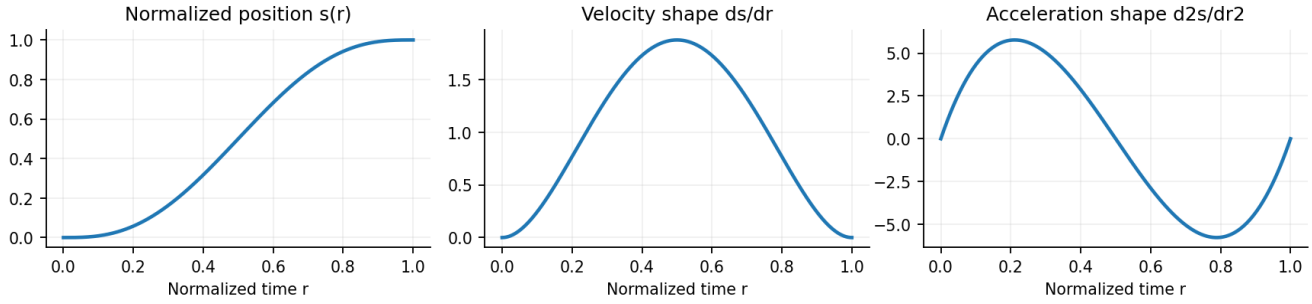


图5-1：归一化五次多项式的位移、速度与加速度形状；多轴共用进度s(r)，并非共用相同角速度。

5.2 运动时间的约束

若考虑每轴最大速度和加速度，不能只选一个“看起来足够长”的T。针对该五次曲线，最大归一化速度系数为1.875，因此至少要求：

$$T \geq \max_i \{ 1.875 \cdot |\Delta q_i| / v_{i,max} \}$$

还应同时校验加速度、Jerk、关节转矩及热负载约束。现有演示固定T=3.0 s，没有实现自动最短时间搜索、带约束的重定时或在线轨迹重规划。

5.3 “同时结束”不是“同时到达”

统一五次曲线可以保证所有期望轨迹在3.25 s结束，但不同惯量、阻尼、饱和和延迟会产生不同实际响应。项目的实际到位判据是：**位置误差小于0.006 rad且速度绝对值小于0.014 rad/s，连续保持40个控制周期**。稳定到位时刻是满足条件的窗口起点，且只在接近规划结束的一段时间内开始统计。

> **解释128 ms的正确方式**
>
> 仿真中的20轴到位时间跨度是128 ms，但不能简单解释为“20轴没有同步执行命令”。它描述的是在特定容差与进入窗口规则下，各关节实际达到稳定终态的时间差。评估同步品质，还应检查归一化进度误差、相位同步误差和末端工艺允许的配合偏差。

CONTROLLER

06 离散PID、前馈与执行器处理

6.1 控制器表达式

$$e[k] = q_d[k] - q[k]$$
$$\tau_{cmd}[k] = K_p e[k] + K_i I[k] + K_d(v_d[k] - v[k]) + \tau_{ff}[k]$$

这里的D项是期望速度减实际速度，避免直接对位置误差差分产生目标跳变时的微分冲击。前馈基于同一简化模型：

$$\tau_{ff} = J \cdot a_d + D \cdot v_d + \tau_c \tanh(v_d/0.025)$$

PID增益采用一个目标二阶闭环频率和阻尼比进行初始化：ω_n=22 rad/s、ζ=0.90、K_i=1.5；因此$K_p=J\omega_n^2$，$K_d=\max(0, 2\zeta\omega_n J - D)$。这只是依据简化模型构造的初始参数，不是通过系统辨识或严格鲁棒稳定性分析获得的参数。

6.2 积分抗饱和

积分状态先被限制在±0.15，然后预测加入积分项后是否会在当前误差方向上继续推动转矩超过上限。如果会进一步饱和，则暂停对应方向的积分更新。最终输出再次做转矩幅值限幅。

```
candidate = clamp(integral + error * Ts, -0.15, 0.15);
trial = P_D_FF + Ki * candidate;
if (!further_saturation) integral = candidate;
torque_cmd = clamp(P_D_FF + Ki * integral,
                   -max_torque, +max_torque);
```

6.3 该PID实现的适用边界

- 仿真已直接提供关节位置与速度，无编码器采样噪声或速度估计问题。
- 积分状态在故障触发时归零，以避免恢复后积累的旧误差导致突发输出。
- 若实际驱动器已经自带位置环/速度环/电流环，不应在不理解其内部结构的情况下叠加相同控制环。
- 生产系统需要明确控制量是电流、转矩、速度还是位置指令；本例假设输出关节转矩。

DELAY & CLOCK

07 通信延迟、序列号与统一控制时钟

7.1 指令为什么要经过延迟队列

在理想仿真中，PID计算完毕就立刻作用到电机，会掩盖通信延迟、乱序等实际问题。本项目为每轴设置32个固定大小的待投递槽位，指令携带序号、执行力矩和计划到达tick。

$$\text{delay}_{ticks} = 1 + (\text{joint_index} \bmod 4) + \text{occasional_extra}(2)$$

基础延迟为1–4 ms，某些时间区间增加2 ms，因此个别命令可能最多延迟6 ms。到达时选择序号更新的命令，避免旧包覆盖新包；队列满时记录丢弃计数，但当前版本不会自动以队列溢出触发保护。

7.2 时间同步与消息同时发送不是一回事

当前20个PID在同一虚拟时刻采样轨迹，但命令按不同延迟到达各自关节。**这并不表示驱动器在同一个硬件时钟沿执行命令**。若改为真实EtherCAT多轴驱动，通常需要支持分布式时钟的从站与同步控制模式；如果是CAN/CANopen，则需结合总线带宽、同步报文和设备自身执行机制设计。EtherCAT DC的原理见参考资料[R3]。

7.3 进一步设计可验证的同步协议

字段	用途	需要验证

sequence	检测乱序、重复、旧指令	递增性与重置规则
send_timestamp	计算发送至接收时延	时钟域一致
execution_time	指定预定生效时刻	从站时钟同步误差
heartbeat_id	检测消息停止更新	超时与伪活跃问题
mode/state	声明控制模式与设备状态	非法状态转移拦截

SAFETY

08　心跳、看门狗与软件安全状态机

8.1 为什么同时需要心跳和命令看门狗?

心跳表示对方的周期性活跃信息仍然更新；命令看门狗检查新的有效控制命令是否持续到达。某进程可能仍能发心跳，但规划线程已经停止输出新命令；也可能控制器挂起，驱动器在本地继续保持最后一次转矩。两种风险需要独立判断。

安全监测	演示阈值	保护动作
控制器心跳超时	50 ms	触发组级SAFE_STOP
各驱动器心跳超时	50 ms	触发组级SAFE_STOP
本地命令看门狗	45 ms	本轴使用阻尼制动并向监督器报告
持续位置跟踪误差	0.065 rad × 60周期	触发组级SAFE_STOP
低速停止确认	全轴速度<0.006 rad/s，持续50 ms	进入STOPPED
模拟制动超时	0.8 s	进入ESTOP软件标签



若SAFE_STOP超过0.8 s仍未满足低速条件，则从SAFE_STOP转入ESTOP标签。该分支只是告警状态，不包含安全继电器、机械制动器或驱动器STO的实际动作。

8.2 触发条件的优先关系

在RUNNING时，程序先检查控制器心跳，再逐轴检查驱动器心跳、命令看门狗、反馈有效性、位置/速度限位和持续跟踪误差。第一次触发条件被记录为停止原因。这里的“优先”来源于代码检查顺序，不应被理解为真实故障严重度的安全等级。

8.3 安全制动模型

$$\tau_{brake} = -\,\mathrm{clamp}(7.5 \cdot \dot{q},\, -\tau_{max},\, \tau_{max})$$

软件监督器将该阻尼转矩应用于所有关节，并考虑转矩斜率限制。其物理效果是使当前运动逐渐衰减，不是固定的恒减速度刹车，也没有考虑机械系统的重力或制动保持能力。

> **真实安全机制不能靠软件标签代替**
>
> 实际机器人若出现重力负载，切断电机转矩可能造成关节下坠。SS1、SS2、STO等功能属于具有明确设备实现和安全要求的驱动级安全功能；如何配置必须依据风险评估、驱动器手册与安全硬件。参考资料[R4]。

EXPERIMENT DESIGN

09 故障注入与试验方法

全部工况均在同样的仿真时长4.5 s、同样20轴参数及同样参考轨迹下运行。故障起始时刻统一设置为1.450 s，目的是让正常和故障数据可以直接对齐比较。

工况	故障注入方法	预期可观测现象
nominal	不注入故障；测量1 ms唤醒迟到量	20轴正常到位、无安全触发
jam	第8轴机械卡住，强制速度为0	参考继续移动，误差累积，跟踪误差保护
controller_heartbeat	1.45–1.75 s暂停控制器心跳与命令更新	中央心跳超时，所有轴模拟减速
drive_heartbeat	1.45–1.75 s暂停第13轴驱动器心跳	单轴心跳超时，所有轴模拟减速

故障反应时间定义

$$\text{检测延迟 } T_{detect} = T_{trigger} - T_{inject}$$
$$\text{模拟停车历时 } T_{stop} = T_{stopped} - T_{trigger}$$

以上定义明确区分“发现异常”和“速度降低到停止判据”。该仿真未定义人员安全距离或安全停机的功能安全等级，因此不能直接拿上述时间判定实际设备是否安全。

RESULTS

10 仿真结果与关键图表

10.1 四工况结果总表

工况	保护触发	STOPPED	检测延迟	减速历时	最终状态
正常	—	—	—	—	RUNNING
第8轴卡滞	1.806 s	1.981 s	356 ms	175 ms	STOPPED
控制器心跳丢失	1.490 s	1.662 s	40 ms	172 ms	STOPPED
第13轴心跳丢失	1.490 s	1.665 s	40 ms	175 ms	STOPPED

心跳丢失场景中，虽然阈值写作50 ms，但触发时间距注入点仅40 ms。这是因为上一条心跳通常在1.440 s更新，1.450 s注入停发，至1.490 s距“最后收到的心跳”正好约50 ms；超时是相对于最后一次心跳计算，而不是相对于注入时刻计算。

10.2 正常工况：位置跟踪误差

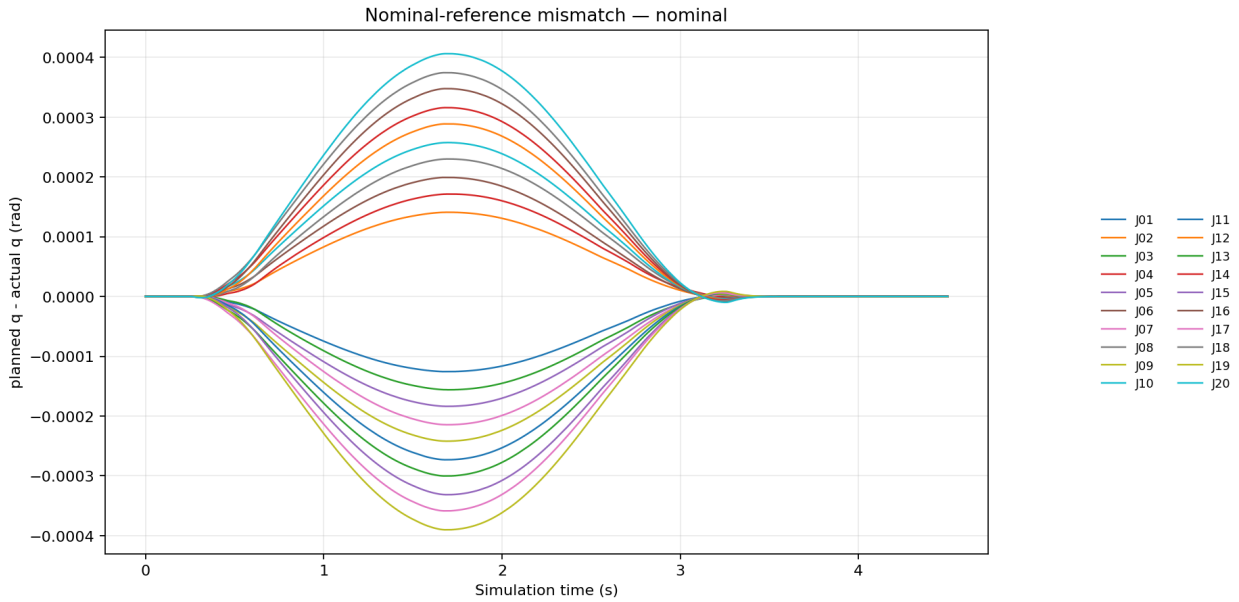


图10-1：20轴在正常工况下的参考-实际位置误差；CSV全时段最大绝对误差0.0004063 rad（约0.0233°）。

20条误差曲线形状相似，幅值随关节目标位移不同而略有差别。这符合当前前馈模型与被控对象模型高度匹配的设定。该结果证明示例实现可在此模型下工作，但并不说明对参数误差、模型不确定性和实物摩擦具有同等级性能。

10.3 正常工况：实际稳定到位时刻

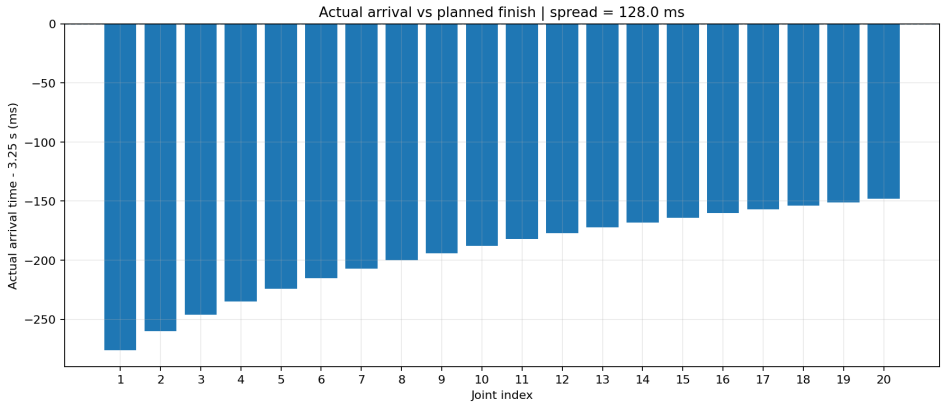
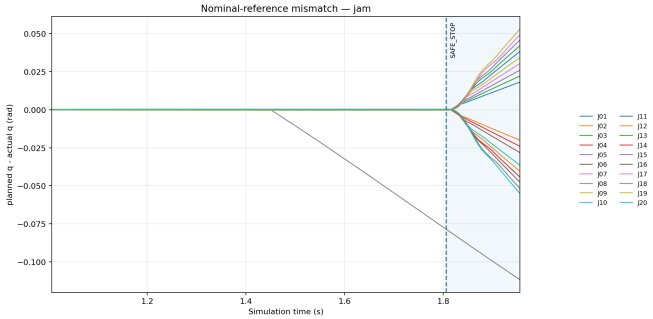


图10-2：每轴稳定到位时刻相对3.25 s规划终点的差值；20个关节均在目标时刻之前满足给定容差，最早与最晚相差128 ms。

因为到位判据采用非零位置/速度容差，关节即使仍处于计划轨迹末端，也可能提前满足“接近静止目标”的条件。要评价真正的运动协调，应补充整段运动的归一化进度误差，而不是只比较首次进入容差的时间。

10.4 第8轴卡滞：误差与停车行为



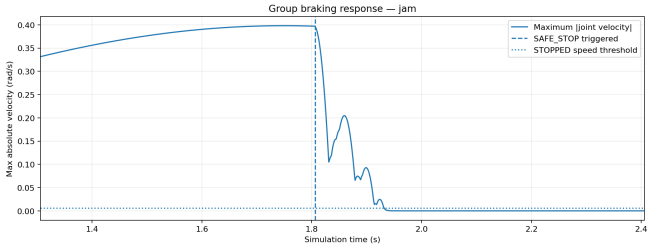


图10-3：第8轴在1.45 s卡滞后误差逐渐增大，于1.806 s触发组停机。

图10-4：组级阻尼制动后的最大关节速度变化。

卡滞模型使第8轴速度直接变为0，但并不禁止电机转矩继续积累。故障检测依赖位置跟踪误差的持续越限，因而存在明显检测延迟。更真实的方案应加入实际电流与目标电流差异、负载估计残差、速度一致性、碰撞与力矩阈值等诊断信号。

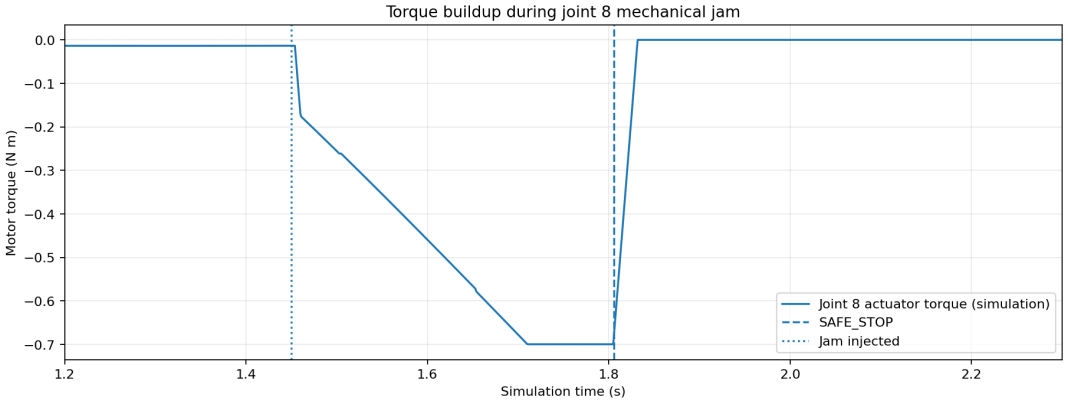


图10-5：卡滞期间模拟的第8轴电机输出转矩变化。该曲线不是驱动器相电流，也不是实际减速器受力测量。

10.5 心跳丢失与看门狗反应

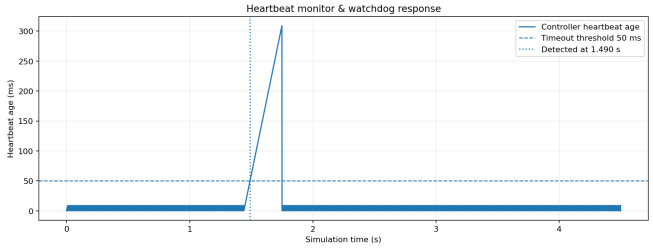


图10-6：中央心跳年龄随停止发送而增长，超过50 ms阈值后触发组级保护。

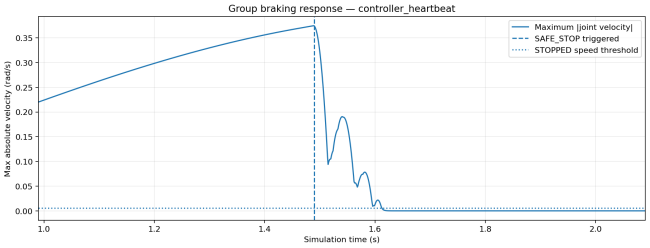


图10-7：控制器心跳超时后所有关节的最大速度被降低至停止门限。

控制器心跳丢失和第13轴驱动器心跳丢失都在1.490 s触发保护，分别于1.662 s和1.665 s进入STOPPED。两组工况表明当前软件逻辑能区分故障源并记录触发时间，但没有验证真实网络失联时驱动器本地功能是否独立有效。

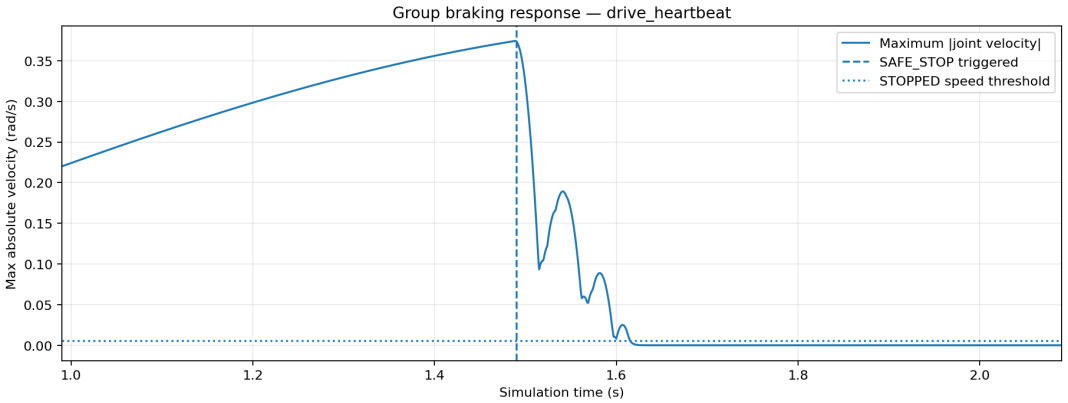


图10-8：第13轴心跳异常后采用同一组级制动策略。

10.6 日志中“故障后误差继续变大”的解释

故障发生后，程序保留原定参考轨迹用于离线分析，但实际执行已经切换为模拟制动。因此故障CSV中的`error_i = q_ref_i − q_i`在停机后可能继续增大；这不代表安全控制仍在追踪原路径，更不能用它评估制动控制器精度。停机效果应看速度、转矩与停止状态日志。

TIMING

11　控制周期抖动与实时性评价

正常实验以 `--realtime` 开启1 ms周期的墙上时间定时，程序使用steady_clock与sleep_until等待预期的唤醒时刻。每次唤醒后记录：

$$\text{唤醒迟到量} = t_{\text{wakeup}} - t_{\text{scheduled}}$$
$$\text{周期误差} = (t_{\text{wakeup},k} - t_{\text{wakeup},k-1}) - 1\ \text{ms}$$

233.855 µs	3854.283 µs	305.080 µs
唤醒迟到量 P99	最大唤醒迟到量	周期绝对偏差 P99

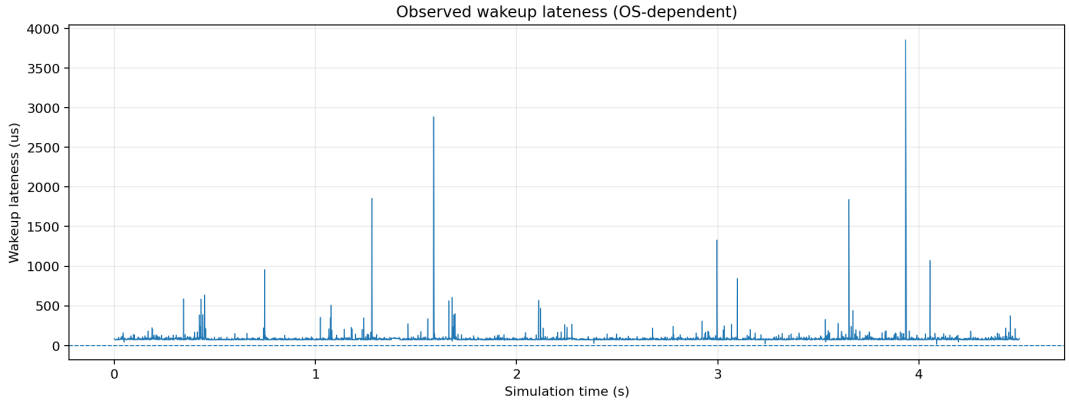


图11-1：正常工况中各周期的调度唤醒迟到量（来自本次运行环境的实际计时）。

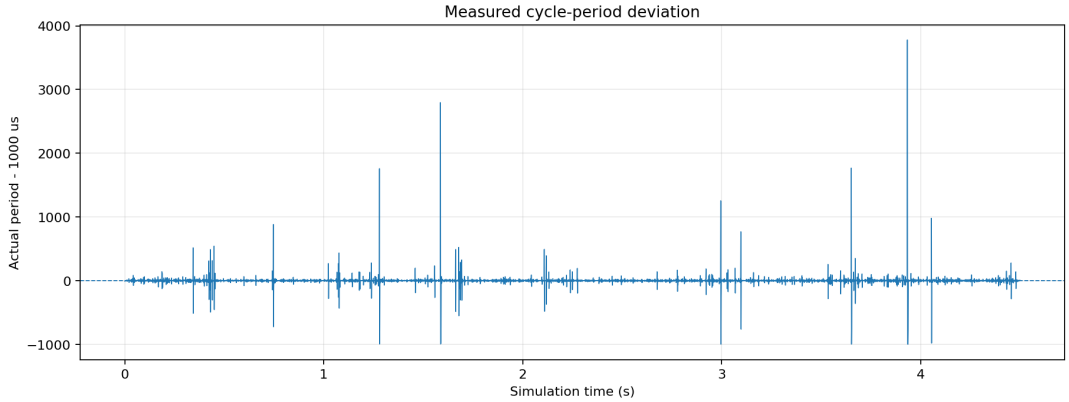


图11-2：相邻唤醒间隔与1 ms目标周期的差；本次出现9次唤醒迟到量超过1 ms，最大绝对周期偏差约3.78 ms。

> **不能声称“硬实时1 kHz”**
>
> 1 ms是程序期望的调度周期；测试中出现毫秒级唤醒超时，说明普通Linux/WSL或当前宿主调度环境没有提供可证明的1 ms最坏情况保证。并且代码测量的是“唤醒时刻”，还没有完整测量控制计算的WCET、执行截止期和总线命令实际生效时刻。参考资料[R1–R2]。

实时化升级要点

- 区分目标周期、实际周期、最坏执行时间(WCET)、最大调度延迟和截止期违约次数。
- 对关键线程采取合适的实时调度、内存锁定和预分配策略；避免阻塞I/O、不可控互斥等待与文件写入。
- 使用绝对时间唤醒而不是相对sleep；Linux `clock_nanosleep(TIMER_ABSTIME)`是可研究的实现方案。
- 把控制线程与CSV持久化分离；并且在硬件目标机而非只在WSL中进行长时间负载测试。

REPRODUCTION

12 项目结构、复现命令与阅读路线

12.1 文件结构

```
servo20_demo/
├── CMakeLists.txt
├── src/main.cpp                  # 20轴模型、PID、通信延迟、安全逻辑
├── tools/plot_results.py         # 根据CSV画图
├── tools/check_results.py        # 工况回归验证
├── run_all.sh                    # 编译、运行、绘图及检查
├── results/                      # 四工况的CSV、事件、汇总
└── plots/                        # 实验图表
```

12.2 Ubuntu / WSL复现

```
sudo apt update
sudo apt install -y build-essential cmake python3-matplotlib
cd servo20_demo
bash run_all.sh

# 可选择单独运行某个场景:
./build/servo20 --scenario nominal --realtime --out results
./build/servo20 --scenario jam --out results
./build/servo20 --scenario controller_heartbeat --out results
./build/servo20 --scenario drive_heartbeat --out results
python3 tools/plot_results.py --input results --output plots
python3 tools/check_results.py
```

重新运行 `--realtime` 后的抖动指标可能与本报告不同，因为它受系统负载和调度环境影响。正常快速仿真不应被用来计算墙上时间抖动；代码对这些指标写入空值或NaN。

12.3 推荐源码阅读顺序

代码符号	要回答的工程问题
`make_config()`	为什么20个关节具有不同动态响应?
`ref_at()`	如何统一运动时间并产生位置/速度/加速度前馈?
`PID::update()`	怎么离散化PID、做前馈和抗积分饱和?
`DelayLine`	如何处理通信延迟、乱序和旧包覆盖?
`Joint::simulate()`	电机负载、转矩饱和、卡滞怎么影响状态?
`run()`	心跳、监督器、状态机如何协调20轴安全停止?

ENGINEERING PATH

13 设计缺口、风险与升级路线

13.1 当前实现与生产级系统的差距

当前实现	主要缺口	推荐工程措施
单线程主循环	整体线程卡住时，监督器也卡住	独立安全任务、真实驱动器本地保护、硬件看门狗
20轴独立刚性模型	没有多关节耦合、重力及接触	加入M(q)、C(q,v)、g(q)及机械约束
合成J/D与前馈完美匹配	低估不确定性和真实跟踪误差	参数扰动、辨识数据、未建模摩擦和噪声

软件延迟队列	没有真实总线争用和从站时钟	CAN负载核算或EtherCAT DC/HIL验证
单一跟踪误差阈值	卡滞检测晚、误报/漏报风险未知	速度残差、电流残差、观测器及多级保护
阻尼型软件停车	不能证明功能安全与重力保持	驱动器安全功能、独立急停和制动器验证

13.2 建议的五阶段迭代路线

1. **M1：测试框架完善。**增加随机种子配置、每关节参数CSV、控制器周期分析和连续回归测试；把阈值与参数抽离成配置文件。
2. **M2：多线程软件架构。**一个实时控制线程、一个规划线程、一个日志线程；用预分配无阻塞通信机制交换数据，测试竞争与丢包。
3. **M3：故障树完善。**增加编码器冻结/跳变、电流反馈异常、命令序号停滞、队列溢出、长延迟、减速器摩擦变化；测量检测延迟分布。
4. **M4：参数辨识。**给各轴加入扫频和PRBS激励模块，估计J、D与摩擦参数；用未参与拟合的数据做交叉验证，并观察参数偏差对跟踪性能的影响。
5. **M5：硬件在环与安全评审。**选择真实控制器与驱动器接口，对齐实际时间戳、测试故障安全状态；由合格工程人员实施风险评估与独立安全保护方案。

13.3 风险优先级（面向后续学习，而非正式FMEA）

风险	后果	优先措施
软件进程整体挂死	软件监督器无法执行	独立设备级看门狗与安全停机链
关节卡死且转矩持续作用	驱动过载或结构损伤	电流/转矩限制、快速残差检测
通信延迟长于控制周期	旧指令滞留、轨迹跟踪变差	同步执行时间、丢弃策略及截止期检查
急停时仍有重力负载	自由下坠或工装损坏	机械制动器/安全功能与重力分析

VALIDATION & REFERENCES

14 验证清单、面试总结与参考资料

14.1 后续验证应留下哪些证据?

测试类别	建议指标	当前项目状态
正常轨迹跟踪	最大/RMS误差；稳态误差；极限工况	已有最大误差与原始CSV
多轴同步	进度同步误差；首次稳定到位差；末端协同误差	仅有稳定到位差128 ms
通信延迟	P50/P99/最大延迟、丢包与乱序率	实现人工延迟，未统计全套指标
故障检测	检测延迟、停车时间、误报/漏报率	四个确定性工况，有单次结果
实时调度	WCET、截止期违约、长时间P99.9	记录唤醒迟到量和周期误差
真实安全系统	设备认证、制动距离、重力保持与故障覆盖	尚未实现或验证

14.2 面试时用90秒解释这个项目

示范回答

我用C++17实现了一个20轴异构伺服控制仿真。每个关节具有不同惯量、阻尼和转矩限制，控制器用离散PID加模型前馈，所有关节共用五次多项式时间参数来协调运动。为了更接近工程系统，我加入了独立命令延迟队列、序列号去旧、控制器与驱动器心跳、命令看门狗和持续跟踪误差保护。正常工况20轴都按给定容差到位，最大误差约0.000406 rad；第8轴卡滞在356 ms后被发现，心跳丢失能在本例设定下触发组级软件停止。我还实测了1 ms定时循环的唤醒抖动，发现存在毫秒级延迟，所以明确把仿真同步、操作系统实时性与真实硬件安全分开讨论。这个项目下一步会重点做真实驱动器通信、独立看门狗及故障检测延迟优化。

14.3 参考资料与项目数据

本报告的公式、数值结果和软件行为以项目文件为第一证据来源；以下公开技术资料只用于解释相关工程原则。

1. **[S1] 源码：** `servo20_demo/src/main.cpp`，包含控制器、关节模型、延迟队列、监督器与统计逻辑。
2. **[S2] 实验数据：** `servo20_demo/results/` 下的四组CSV、事件日志及summary文本；图表来自 `servo20_demo/plots/`。
3. **[R1] ROS 2 Real-time Background：** https://design.ros2.org/articles/realtime_background.html，讨论实时路径中的内存分配、阻塞I/O与页错误。
4. **[R2] Linux clock_nanosleep(2)：** https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html，绝对时间定时与调度延迟的说明。
5. **[R3] Beckhoff EtherCAT Distributed Clocks：** https://www.beckhoff.com/en-en/products/i-o/ethercat/，分布式时钟及多轴同步原理。
6. **[R4] Siemens Safety Integrated Commissioning Manual：** https://support.industry.siemens.com/cs/attachments/109816998/MC_SI_commiss_man_0123_en-US.pdf，驱动器STO、SS1等安全功能的工程实现背景。

最终结论

在这一组确定性软件仿真中，多轴统一轨迹、PID与前馈、命令延迟、心跳与看门狗的逻辑均得到了可复现的行为验证；但真实电机、电流环、硬件时间同步、机械安全和功能安全认证都不在验证范围内。因此，它适合作为工程面试展示与后续HIL开发的基础，而不能直接部署到实物机器人。

说明：报告中的安全阈值均为学习演示参数；图表采自本项目生成的数据。若重新编译或重新运行，特别是实时调度测量，数字可能发生变化，报告结果不自动代表新一次运行。